

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

FELIPE RAFAEL NIENDICKER
RA: 11231101151

LOGIN SEGURO
AUTENTICAÇÃO, AUTORIZAÇÃO E SESSÕES COM SPRING BOOT E
MONGODB

MOGI DAS CRUZES
2026

FELIPE RAFAEL NIENDICKER

RA: 11231101151

**LOGIN SEGURO
AUTENTICAÇÃO, AUTORIZAÇÃO E SESSÕES COM SPRING BOOT E
MONGODB**

Trabalho acadêmico apresentado à disciplina de Aplicativos Web do curso de Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes, sob orientação do professor Alessandro Aparecido da Silva Horas.

MOGI DAS CRUZES

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma aplicação web de cadastro, autenticação e autorização de usuários. O sistema utiliza Java 17, Spring Boot, Spring Security, Thymeleaf e MongoDB Atlas. As senhas são protegidas por hash BCrypt, e as sessões HTTP são configuradas para persistência no MongoDB. O controle de acesso contempla os perfis USUARIO, MODERADOR e ADMIN, com restrições específicas para páginas e endpoints. A interface utiliza templates e fragmentos reutilizáveis, com estilos separados da lógica de negócio para facilitar futuras personalizações visuais. A suíte de testes contabilizou 41 testes: 40 executados com sucesso e um teste de integração com o Atlas ignorado por exigir ativação explícita. O documento descreve a arquitetura, as decisões de segurança, a integração com o banco de dados e os procedimentos de execução e teste.

Palavras-chave: autenticação; Spring Security; MongoDB Atlas; Thymeleaf; controle de acesso.

SUMÁRIO

1 INTRODUÇÃO 1

2 OBJETIVOS 1

2.1 Objetivo geral 1

2.2 Objetivos específicos 1

3 TECNOLOGIAS UTILIZADAS 2

4 ARQUITETURA DO SISTEMA 2

5 SEGURANÇA E AUTENTICAÇÃO 3

5.1 Cadastro e validação 3

5.2 Proteção das senhas 3

5.3 Login, sessão e logout 3

5.4 Proteção CSRF 4

6 CONTROLE DE ACESSO POR PERFIS 4

7 INTEGRAÇÃO COM O MONGODB ATLAS 4

8 INTERFACE THYMELEAF E PREPARAÇÃO PARA TEMAS 5

9 TESTES REALIZADOS 5

10 CONSIDERAÇÕES FINAIS 6

REFERÊNCIAS 6

1 INTRODUÇÃO

O Login Seguro foi desenvolvido para a disciplina de Aplicativos Web. A aplicação reúne os fluxos de cadastro, login e logout, o controle de acesso por perfis e a persistência das sessões de usuários.

A aplicação utiliza Java 17, Spring Boot, Spring Security, Thymeleaf e MongoDB Atlas. As senhas são processadas com BCrypt antes da gravação, e o navegador mantém apenas o identificador da sessão em um cookie. Os dados dos usuários e das sessões permanecem no MongoDB.

O sistema disponibiliza uma API REST e uma interface web renderizada no servidor. As duas formas de acesso utilizam as mesmas regras de autenticação e autorização. Essa organização permitiu demonstrar os recursos de segurança sem misturar a apresentação visual com as regras de negócio.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver uma aplicação web para demonstrar, de forma prática, autenticação, autorização e persistência de sessões com tecnologias do ecossistema Spring.

2.2 Objetivos específicos

- Cadastrar usuários com validação de nome, e-mail e senha.
- Normalizar o e-mail e impedir cadastros duplicados.
- Armazenar a senha somente como hash BCrypt.
- Autenticar usuários por e-mail e senha.
- Persistir sessões HTTP no MongoDB.
- Invalidar a sessão durante o logout.
- Proteger requisições de alteração com CSRF.
- Restringir recursos conforme os perfis *USUARIO*, *MODERADOR* e *ADMIN*.
- Disponibilizar páginas Thymeleaf responsivas.
- Separar o código Java, os templates HTML e os estilos CSS.
- Organizar o desenvolvimento com Gitflow.

3 TECNOLOGIAS UTILIZADAS

O Java 17 é a plataforma de execução do projeto (ORACLE, [s. d.]). O Spring Boot 4.1.1 configura a aplicação e gerencia as versões das principais dependências.

O Spring Security autentica os usuários, mantém o contexto de segurança na sessão, aplica as permissões e valida os tokens CSRF. O Spring Data MongoDB fornece o repositório usado para gravar e consultar usuários.

O MongoDB Atlas armazena os documentos da aplicação. A biblioteca MongoDB Spring Session 4.0.0 integra as sessões HTTP ao MongoDB, evitando que o estado de autenticação permaneça somente na memória do processo (MONGODB, [s. d.]).

O Thymeleaf processa os templates HTML no servidor e integra os formulários ao Spring MVC. A integração permite associar os campos aos objetos Java e exibir mensagens de validação (THYMELEAF, [s. d.]).

O Maven Wrapper executa a compilação e os testes sem exigir uma instalação global do Maven. Os testes utilizam JUnit, Mockito, MockMvc e Spring Security Test.

4 ARQUITETURA DO SISTEMA

O código está dividido em responsabilidades simples:

- *config* contém as configurações de segurança e sessões.
- *controller* recebe as requisições REST e apresenta as páginas Thymeleaf.
- *dto* define os dados aceitos e devolvidos pelas APIs.
- *service* concentra o cadastro e o carregamento dos usuários.
- *repository* acessa o MongoDB por meio do Spring Data.
- *model* representa o documento *Usuario*.
- *templates* contém as páginas e os fragmentos HTML.
- *static/css* contém os estilos visuais.

O documento *Usuario* possui identificador, nome, e-mail, hash da senha e perfil. Os usuários ficam na coleção *usuarios*. O *UsuarioRepository* estende *MongoRepository* e oferece a consulta por e-mail.

A API REST e a interface web reutilizam os mesmos serviços e a mesma configuração de segurança. O filtro de autenticação do Spring Security processa o formulário HTML de login. A API de login recebe JSON e realiza a autenticação por meio do *AuthenticationManager*. Essa separação mantém os dois contratos de entrada sem duplicar a regra de verificação das credenciais.

5 SEGURANÇA E AUTENTICAÇÃO

5.1 Cadastro e validação

O cadastro recebe nome, e-mail e senha. Os campos são obrigatórios, o e-mail deve ter formato válido e a senha precisa conter pelo menos oito caracteres. Antes da consulta e da gravação, o sistema remove os espaços externos do e-mail e converte seus caracteres para letras minúsculas.

A aplicação consulta o e-mail antes de salvar o usuário. O campo também possui um índice único no MongoDB, que protege contra duplicidades causadas por requisições simultâneas.

O cadastro público não recebe o perfil como parâmetro. O serviço atribui *USUARIO* a toda nova conta, impedindo a criação pública de usuários privilegiados.

5.2 Proteção das senhas

O *BCryptPasswordEncoder* gera o hash armazenado no documento do usuário. A senha original não é gravada. Durante o login, o Spring Security compara a senha informada com o hash por meio da interface *PasswordEncoder* (SPRING SECURITY, [s. d.]).

As respostas das APIs utilizam um DTO com identificador, nome, e-mail e perfil. Esse DTO não possui campo de senha. Os DTOs de entrada também substituem senha e token por marcadores protegidos em sua representação textual, reduzindo a exposição acidental em logs.

5.3 Login, sessão e logout

O login normaliza o e-mail e delega a validação das credenciais ao Spring Security. Depois da autenticação, o contexto de segurança é associado à sessão HTTP e salvo pelo *SecurityContextRepository*.

A aplicação altera o identificador da sessão no momento da autenticação. Essa medida reduz o risco de fixação de sessão. O cookie recebe o nome *SESSION* e utiliza *HttpOnly* e *SameSite=Lax*. Em uma execução com HTTPS, a variável *SESSION_COOKIE_SECURE=true* ativa o atributo *Secure*. A sessão expira após 30 minutos de inatividade.

O logout aceita requisições *POST*, exige um token CSRF válido, invalida a sessão, limpa a autenticação e remove o cookie. A proteção de login e logout contra CSRF segue a orientação do Spring Security para aplicações baseadas em sessão (SPRING SECURITY,

[s. d.]b).

5.4 Proteção CSRF

O CSRF permanece habilitado em toda a aplicação. A API fornece o token em `/api/auth/csrf`. Os formulários Thymeleaf enviam o mesmo dado em um campo oculto. Nas requisições JSON, o cliente envia o cookie `XSRF-TOKEN` e o valor no cabeçalho `X-XSRF-TOKEN`. Uma requisição `POST` sem token válido recebe HTTP 403.

6 CONTROLE DE ACESSO POR PERFIS

O `UsuarioDetailsService` transforma o perfil armazenado no MongoDB em uma autoridade do Spring Security com o prefixo `ROLE_`. O sistema trabalha com as autoridades `ROLE_USUARIO`, `ROLE_MODERADOR` e `ROLE_ADMIN`.

Recurso	USUARIO	MODERADOR	ADMIN
<code>/painel/usuario</code>	Sim	Sim	Sim
<code>/painel/moderador</code>	Não	Sim	Sim
<code>/painel/admin</code>	Não	Não	Sim
<code>/api/auth/me</code>	Sim	Sim	Sim

Uma requisição não autenticada para uma API protegida recebe HTTP 401. Quando o usuário está autenticado, mas não possui a autoridade exigida, a API responde com HTTP 403. Na interface web, a mesma situação encaminha o usuário para a página de acesso negado.

7 INTEGRAÇÃO COM O MONGODB ATLAS

A aplicação lê a conexão do MongoDB por meio da variável de ambiente `MONGODB_URI`. A URI real não faz parte do código-fonte. O `.env.example` apresenta somente um formato fictício, enquanto o `.gitignore` impede o versionamento de arquivos `.env` locais.

O usuário configurado no Atlas deve ter permissões restritas ao banco `login_seguro`. O controle de acesso de rede do Atlas limita os endereços que podem abrir conexões. Quando a senha contém caracteres especiais, seu valor precisa ser codificado para uso na URI.

Os usuários são gravados na coleção `usuarios`. A anotação `@EnableMongoHttpSession` direciona as sessões para a coleção `sessoes`. Assim, o servidor pode recuperar a autenticação em requisições posteriores sem manter todo o estado apenas

em memória. A integração do Spring Session com MongoDB também permite compartilhar as sessões entre instâncias compatíveis da aplicação (MONGODB, [s. d.]).

O projeto inclui o teste opcional *AtlasConnectionIntegrationTest*. Ele cria um documento com identificador aleatório na coleção exclusiva *teste_conexao_atlas*, consulta os dados e remove somente o documento criado. O teste exige ativação explícita e permanece ignorado no comando Maven padrão.

8 INTERFACE THYMELEAF E PREPARAÇÃO PARA TEMAS

A interface possui páginas de login, cadastro, painéis dos três perfis e acesso negado. Cabeçalho, navegação e rodapé são fragmentos reutilizáveis. Os templates ficam em *templates*, enquanto o estilo visual fica em *static/css*.

O arquivo CSS declara propriedades personalizadas no seletor *:root* para cores, fonte, espaçamento, raio de borda e sombra. Uma nova identidade visual pode alterar essas variáveis sem interferir na autenticação ou nas regras de negócio. O CSS também adapta o layout para telas menores.

Os templates exibem apenas nome, e-mail e perfil. Senha e hash não são adicionados ao modelo das páginas. Os formulários utilizam *th:action*, *th:field* e mensagens de erro da integração entre Thymeleaf e Spring MVC (THYMELEAF, [s. d.]).

9 TESTES REALIZADOS

Os testes locais usam repositórios e operações MongoDB simulados, sem conexão com o Atlas. A suíte verifica:

- cadastro válido, normalização de e-mail, BCrypt e perfil padrão;
- validação dos dados e tratamento de e-mail duplicado;
- login correto, senha incorreta e usuário inexistente;
- criação e invalidação da sessão simulada;
- consulta do usuário autenticado;
- exigência do token CSRF;
- ausência de senha e hash nas respostas;
- permissões e bloqueios dos três perfis;
- páginas públicas e restritas;
- cadastro, login, logout e redirecionamento pela interface;
- renderização dos templates Thymeleaf;

- atributos do cookie e configuração da coleção de sessões;
- carregamento do contexto Spring sem acesso ao Atlas.

O comando `.!mvnw.cmd clean test` contabilizou 41 testes, sem falhas ou erros. Quarenta testes foram executados com sucesso. O único teste ignorado foi o *AtlasConnectionIntegrationTest*, pois sua execução depende de solicitação explícita e de uma variável local com a conexão do Atlas.

10 CONSIDERAÇÕES FINAIS

O Login Seguro reúne cadastro, autenticação por senha e autorização por perfis em uma aplicação pequena, com responsabilidades separadas entre controllers, serviços, repositórios e templates. O cadastro público sempre cria o perfil *USUARIO*, enquanto o Spring Security restringe os painéis de moderação e administração.

As senhas permanecem protegidas por BCrypt, e as respostas não expõem o valor original nem o hash. O uso de sessões HTTP mantém a autenticação no servidor, e a integração com o MongoDB permite persistir esse estado fora da memória da aplicação. CSRF, cookies *HttpOnly* e a alteração do identificador da sessão complementam as medidas adotadas.

A interface Thymeleaf cobre o fluxo principal sem depender de um framework JavaScript. A separação dos templates, estilos e regras de negócio facilita a leitura do projeto e permite modificar o tema visual sem alterar a segurança. Os testes automatizados verificam os principais comportamentos localmente e mantêm o acesso ao Atlas separado em um teste de integração opcional.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724:2024: informação e documentação — trabalhos acadêmicos — apresentação**. Rio de Janeiro: ABNT, 2024.

MONGODB. **Spring Session MongoDB Integration**. [S. l.], [s. d.]. Disponível em: <<https://www.mongodb.com/docs/drivers/java/sync/current/integrations/spring-session/>>. Acesso em: 8 out. 2026.

ORACLE. **JDK 17 Documentation**. [S. l.], [s. d.]. Disponível em: <<https://docs.oracle.com/en/java/javase/17/>>. Acesso em: 8 out. 2026.

SPRING. **Spring Boot 4.1.1 Reference Documentation**. [S. l.], [s. d.]. Disponível em: <<https://docs.spring.io/spring-boot/documentation.html>>. Acesso em: 8 out. 2026.

SPRING SECURITY. **Password Storage**. [S. l.], [s. d.]a. Disponível em: <<https://docs.spring.io/spring-security/reference/7.0/features/authentication/password-storage.html>>. Acesso em: 8 out. 2026.

SPRING SECURITY. **Cross Site Request Forgery (CSRF)**. [S. l.], [s. d.]b. Disponível em: <<https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html>>. Acesso em: 8 out. 2026.

SPRING SECURITY. **Authentication Persistence and Session Management**. [S. l.], [s. d.]c. Disponível em: <<https://docs.spring.io/spring-security/reference/servlet/authentication/session-management.html>>. Acesso em: 8 out. 2026.

THYMELEAF. **Tutorial: Thymeleaf + Spring**. Versão 3.1. [S. l.], [s. d.]. Disponível em: <<https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html>>. Acesso em: 8 out. 2026.